TP Unidad 5 — Parte 2: Calidad del software

Metodología de Sistemas II · Tecnicatura Universitaria en Programación · 2026

Equipo Naft: Nahuel Alem · Fabricio Ceniquel Thompson

Proyecto: Sistema Integral de Gestión — Centro Educativo «Educar para Transformar»

Repositorio: https://github.com/fabrithompson/CentroEducativo · **Producción:** https://transformar-para-educar.up.railway.app

1. Funcionalidad y requerimiento seleccionados

Funcionalidad: Acceso de padres (RF-03). Es distinta de la del TP de la Unidad 4, que fue el alta de alumnos (RF-01).

Requerimiento verificable: *«Un padre, madre o tutor sólo puede ver la información de sus propios hijos.»*

Es una de las reglas de negocio críticas del sistema: protege datos de menores (ficha, domicilio, cursada, cuotas). Se puede verificar con un resultado binario, sin interpretación: el selector muestra exactamente a los hijos vinculados, y un pedido sobre otro alumno se rechaza. Además es la regla que más fácil se rompe sin que nadie lo note: alcanza con que una consulta nueva olvide filtrar por tutor.

> ***Por qué no Deportes (RF-02).*** *Era la candidata natural («máximo dos deportes, sin choque de horarios»), pero la inscripción a deportes no tiene pantalla en la web: sólo existe en la API. Un caso de prueba sobre la interfaz no se podía ejecutar. La regla de los dos deportes sí está verificada, pero por las pruebas de integración contra la base.*

2. Caso de prueba

Campo de caso de prueba	**Descripción**
ID de caso de prueba	**CP-RF03-01**
Requerimiento	**RF-03 — Acceso de padres.** Los padres sólo pueden ver y gestionar la información de sus propios hijos.
Prueba diseñada por	Fabricio Ceniquel Thompson
Fecha de diseño de la prueba	26/10/2026
Prueba ejecutada por	Fabricio Ceniquel Thompson (automatizada, Selenium con Microsoft Edge) · GitHub Actions (automatizada, Selenium con Google Chrome, en cada PR)
Fecha de ejecución de la prueba	26/10/2026
Condición previa	Pila completa levantada con los datos del seed (`pnpm --filter backend qa` → http://127.0.0.1:4599). Usuarios `mgomezp` (Mariana Gómez, madre de María Paz Gómez e Ignacio Flores) y `pmedina` (Patricia Medina, madre de Mateo Medina), contraseña `123456`. Navegador sin sesión iniciada.

Pasos de prueba	**Resultados esperados**
1. Abrir http://127.0.0.1:4599/.	Se muestra el portal institucional con el enlace «Iniciar sesión» en la barra superior.

Pasos de prueba	Resultados esperados
2. Hacer clic en «Iniciar sesión».	Se abre el modal «Iniciar Sesión» con los campos *Usuario* y *Contraseña* y el botón «Ingresar al Campus».
3. Escribir `mgomezp` en *Usuario*, `123456` en *Contraseña* y hacer clic en «Ingresar al Campus».	El sistema redirige a `panel_padre.html` (panel de tutores).
4. Desplegar el selector «Alumno a cargo».	Muestra **exactamente dos** opciones: «Gómez, María Paz (6to Grado "A")» y «Flores, Ignacio Tomás (6to Grado "A")». No aparece ningún otro alumno.
5. Cerrar sesión e ingresar como `pmedina` / `123456`.	El selector muestra **una sola** opción: «Medina, Mateo Nicolás (1er Año "A")».
6. Con la sesión de `mgomezp`, pedir los deportes de Mateo escribiendo la ruta de la API a mano (`/api/padres/mis-hijos/{id de Mateo}/deportes`).	La API responde **404** («Recurso no disponible en el portal de tutores.») y no devuelve ningún dato de Mateo. Es 404 y no 403 a propósito: no confirma que ese alumno exista.

Resultado obtenido: ✓ Pasó. Los seis pasos están automatizados con Selenium (sección 6), que los ejecuta en un navegador real tal como los haría una persona: 3 de 3 pruebas en verde con Microsoft Edge en la máquina de desarrollo y con Google Chrome en GitHub Actions, en los PR #41 y #42.

3. Prueba estática: revisión por pares (*peer review*)

En la revisión por pares, un integrante evalúa el trabajo del otro antes de integrarlo para detectar defectos sin ejecutar el código. En este equipo de dos, la regla del plan de trabajo es cruzada: lo que escribe uno lo revisa el otro.

3.1 Buenas prácticas que debe cumplir el código para agilizar la revisión

Práctica	Por qué agiliza	Cómo se cumple en el proyecto
PR chicos y de un solo tema	Un revisor encuentra más defectos en 200 líneas que en 2000; pasadas las 400, la atención cae.	Mediana histórica de 150 líneas por PR. Un issue por PR.
Formato y estilo resueltos por herramientas	El revisor no gasta tiempo en comillas, sangrías o `var`: se concentra en la lógica.	ESLint en el editor, en el pre-commit y en el CI; `.editorconfig`.
CI en verde antes de pedir revisión	No se revisa algo que no compila o rompe pruebas.	Typecheck, lint, integración, móvil y E2E como chequeos obligatorios.

Práctica	Por qué agiliza	Cómo se cumple en el proyecto
Nombres que dicen qué es cada cosa	Se entiende sin leer la implementación.	Vocabulario del dominio en español: `alumnosPorMateria`, `esResponsableFacturacion`, `idsDeMisHijos`.
Funciones cortas y capas separadas	Se revisa cada pieza por separado.	Rutas → servicios → Prisma; las reglas críticas viven en la base (disparadores, `CHECK`).
Comentarios que explican el porqué	El *qué* lo dice el código; el *porqué* es lo que el revisor no puede adivinar.	Ver, por ejemplo, el porqué del 404 en lugar del 403 en el módulo `padres`.
Commits descriptivos y PR documentado	El revisor sabe qué buscar y dónde.	Conventional Commits (validado por hook); plantilla de PR con qué cambia, cómo se probó e issue vinculado.
Pruebas que acompañan el cambio	Muestran qué se quiso garantizar.	Cada regla de negocio tiene su prueba; RF-03, además, su E2E.
Sin secretos, código muerto ni datos reales	Evita que el revisor tenga que buscarlos.	`.env` fuera del repo; datos de prueba del seed.

3.2 Estrategias para aplicar la técnica

1. **Plantilla de PR con lista de control** (`.github/pull_request_template.md`). El revisor marca cada punto: hace lo que dice el issue y nada más, nombres claros, reglas validadas en la base, un padre no ve datos ajenos, sin secretos, migración si cambió el esquema, bitácora de IA al día.
2. **Asignación automática del revisor** con `CODEOWNERS`: GitHub pide la revisión de los dos integrantes al abrir el PR.
3. **Nada entra sin PR**: la protección de `main` rechaza el push directo y exige los seis chequeos del CI.
4. **Orden de lectura fijo**: (1) criterios de aceptación del issue, (2) seguridad y reglas de negocio (RF-03, pagos), (3) pruebas, (4) legibilidad.
5. **Sesiones acotadas**: hasta 400 líneas o 60 minutos por sesión; si el PR es más grande, se pide partirlo.
6. **Comentarios clasificados** en el PR: *bloqueante*, *sugerencia* o *pregunta*. Sólo los bloqueantes impiden aprobar.
7. **Medir la revisión**: defectos encontrados por PR y tiempo de revisión. La línea base del TP de la Unidad 4 mostró 0 de 25 PR con revisión: ese es el número a mejorar.

4. Documentación del proyecto

4.1 Historia de usuario

> *En el repositorio, las historias figuran como HU1 a HU8 asociadas a los requerimientos RF-01 a RF-08 (plan de trabajo, §7). Ésta es la HU3, escrita en formato completo para este caso de prueba. Está cargada como plantilla en `.github/ISSUE_TEMPLATE/historia_de_usuario.yml`.*

HU3 — Acceso de padres (RF-03) · Responsable del modelado: Nahuel Alem

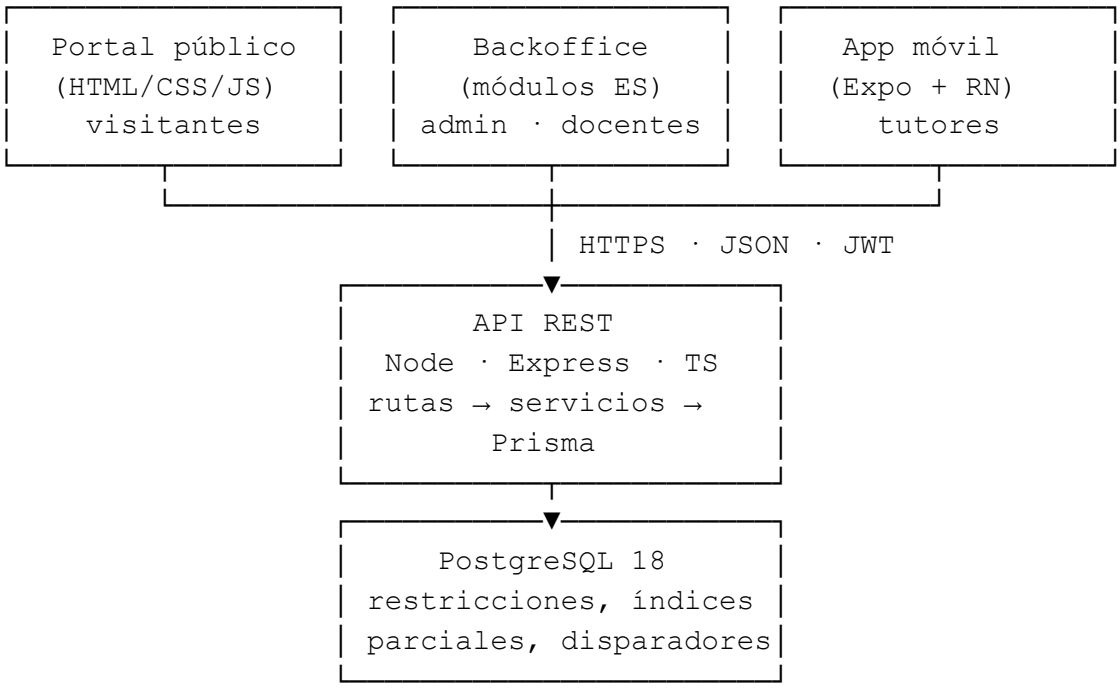
Como madre, padre o tutor de un alumno del colegio, **quiero** consultar en el campus la información de mis hijos (ficha, materias con su profesor, deportes, servicios y cuotas), **para** seguir su escolaridad y sus pagos sin tener que ir a la secretaría.

Criterios de aceptación:

1. Dado un tutor con hijos vinculados, cuando ingresa al campus, el selector «Alumno a cargo» muestra exactamente a sus hijos.
2. Dado un tutor con más de un hijo, puede cambiar de hijo desde el selector y ve la información de ese hijo.
3. Para cada hijo, ve las materias del curso con el profesor a cargo de cada una.
4. Dado un tutor que pide —por la interfaz, por la URL o por la API— datos de un alumno que no es su hijo, el sistema responde 404 y no revela si ese alumno existe.
5. El vínculo tutor ↔ alumno sólo lo crea un administrador, y sólo con un usuario de rol PADRE (lo exige un disparador en la base).

4.2 Arquitectura de la aplicación

Tres aplicaciones —portal web institucional, backoffice de gestión y aplicación móvil para tutores— que comparten **un único backend y una única base de datos**.



Capa	**Tecnología**	**Detalle**
Frontend web	HTML5, CSS3, JavaScript con módulos ES	Lo sirve el mismo backend con `express.static`; sin paso de compilación
App móvil	Expo SDK 57, React Native 0.86	Sólo para tutores
API	Node.js 22, Express 4, TypeScript 5 (estricto), Zod	15 módulos de dominio bajo `src/modules/`
Tiempo real	Socket.IO	Anuncios, notas y avisos sin recargar
Acceso a datos	Prisma 6	12 migraciones versionadas
Base de datos	PostgreSQL 18 en Railway	Las reglas críticas viven en el motor: índice único parcial (máximo dos deportes), disparadores PL/pgSQL (cuatro recorridos fijos, tutor con rol PADRE, recálculo de facturas)
Autenticación	JWT de acceso + cookie de refresco `httpOnly` con revocación	Contraseñas con bcrypt; cambiar la contraseña cierra todas las sesiones
Autorización	Matriz por rol en `shared/authz.ts` + tabla `TutorAlumno`	RF-03 se resuelve en el backend, no en la pantalla
Despliegue	Railway, con infraestructura como código	Cada merge a `main` se construye y despliega solo, después del CI

Regla de dependencia: `HTTP → Aplicación → Dominio → Persistencia`. El dominio no importa Express ni Prisma, así que sus reglas se prueban sin levantar un servidor.

Cómo se resuelve RF-03: cada ruta de `/api/padres/mis-hijos/:alumnoId/...` pasa por un middleware que busca el vínculo en `TutorAlumno` para el tutor autenticado. Si no existe, corta con 404 antes de tocar los datos del alumno.

4.3 Información adicional

Usuarios de prueba (datos del seed, contraseña `123456`, sólo en entornos locales y de prueba):

Rol	Usuario	Uso en las pruebas
Administrador	`fabriynahuel`	Alta de alumnos y vínculos
Tutor con dos hijos	`mgomezp`	Selector de hijos, RF-03
Tutor con un hijo	`pmedina`	RF-03
Docente	`mlopez`	Notas y actividades
Estudiante	`mmedina`	Panel del estudiante

Cómo correr cada prueba:

Qué	Comando	Cantidad
Pruebas del backend y reglas en el motor	`pnpm --filter backend test:integracion`	295 + 17
Pruebas de la app móvil	`pnpm --filter mobile test`	70
Pruebas E2E con Selenium	`pnpm --filter backend test:e2e`	3
Pruebas de humo por HTTP	`pnpm --filter backend test:humo`	4 circuitos
Lint	`pnpm lint`	136 archivos

5. Controles de calidad configurados

Son cuatro capas, de la más cercana a quien escribe el código a la más lejana. Todas quedaron funcionando el 26/10/2026 y están documentadas en `docs/tp5/calidad.md`.

Capa	Herramienta	Cuándo corre	Qué bloquea
Editor	ESLint (extensión de VS Code)	Mientras se escribe	Nada: subraya y corrige al guardar
Commit local	Husky + lint-staged	`git commit`	El commit
Pull request	GitHub Actions + protección de `main`	Cada push a un PR	El merge

Capa	Herramienta	Cuándo corre	Qué bloquea
Análisis central	SonarCloud	Cada PR y cada push a `main`	El merge, por *quality gate* (al activarse)

5.1 Linter en el IDE: ESLint

- **Configuración única** para todo el monorepo en `eslint.config.mjs` (ESLint 9, *flat config*). TypeScript con `typescript-eslint` en el backend y la app; JavaScript plano en el navegador para el frontend; Node para los scripts.
- **Reglas**: las recomendadas de ESLint y de typescript-eslint, más `eqeqeq` (siempre ===), `no-var` y `prefer-const` como **errores**. `no-explicit-any` y variables sin usar quedan como **advertencias**: son deuda del código anterior y se bajan de a poco, sin frenar la adopción.
- **En el editor**: `.vscode/extensions.json` recomienda la extensión de ESLint al abrir el proyecto, y `.vscode/settings.json` corrige al guardar (`source.fixAll.eslint`). `.editorconfig` fija sangría, fin de línea y codificación para cualquier editor.
- **Resultado de la adopción**: el primer pase encontró **36 errores** en 132 archivos. Treinta y tres eran globales que la configuración todavía no conocía; uno era un defecto real (un `let` que nunca se reasignaba en el TOTP de la app móvil), que se corrigió. Hoy: **0 errores y 27 advertencias** en 136 archivos (32,4 KLOC).

```
$ pnpm lint
✗ 27 problems (0 errors, 27 warnings)
```

5.2 Pre-commit hooks: Husky

Se instalan solos con `pnpm install` (script `prepare`).

- **`pre-commit`**: corre ESLint sobre los archivos en *stage* (lint-staged) y `git diff --cached --check`, que rechaza espacios al final de línea y marcadores de conflicto olvidados.
- **`commit-msg`**: exige Conventional Commits (`tipo(ámbito): descripción`).

Evidencia — un commit con `var` y == y otro con un mensaje genérico, rechazados en la máquina local:

```
$ git commit -m "feat: prueba del hook"
✗ eslint --no-warn-ignored:
web\backend\src\demo-hook.ts
  1:1   error  Unexpected var, use let or const instead  no-var
  2:14  error  Expected '===' and instead saw '=='       eqeqeq
✗ 2 problems (2 errors, 0 warnings)
husky - pre-commit script failed (code 1)

$ git commit -m "cambios varios"
✗ El mensaje del commit no sigue Conventional Commits:
    cambios varios
  Formato:  tipo(ámbito): descripción
husky - commit-msg script failed (code 1)
```

5.3 Políticas de Pull Requests

- **Protección de `main`** (activa desde el 26/10/2026): no se puede pushear directo; todo entra por PR; para mergear tienen que estar en verde los seis chequeos (*Typecheck*, *Pruebas de integración (PostgreSQL)*, *Pruebas de la app móvil*, *Lint (ESLint)*, *Pruebas E2E (Selenium)* y *SonarCloud*); las conversaciones deben estar resueltas; sin *force push* ni borrado de la rama; y la regla también vale para los administradores.
- **`CODEOWNERS`**: pide la revisión de `@fabrithompson` y `@Nehuelsito` en cada PR.
- **Plantilla de PR** con issue vinculado (`Closes #N`), cómo se probó y la lista de revisión por pares.
- **Aprobación obligatoria**: por ahora se dejó en 0 para no bloquear la entrega si un integrante no está disponible. El paso siguiente es exigir una aprobación del otro integrante.

5.4 Orquestador centralizado: SonarCloud

- `sonar-project.properties` define el proyecto: separa fuentes de pruebas y excluye migraciones, dependencias y subidas.
- El job `SonarCloud` del CI corre el análisis en cada PR y en cada push a `main`. Mientras no exista el secreto `SONAR_TOKEN`, termina en verde con un aviso, para que el pipeline no quede en rojo por falta de configuración.
- **Activación** (requiere iniciar sesión con la cuenta de GitHub del dueño del repositorio, por eso queda a cargo del equipo): sonarcloud.io → *Log in with GitHub* → *Analyze new project* → `CentroEducativo` → desactivar *Automatic Analysis* → copiar el token → GitHub → *Settings* → *Secrets* → *Actions* → `SONAR_TOKEN`. Desde ese momento el *quality gate* aparece como chequeo en cada PR.

6. Prueba dinámica: Selenium WebDriver

6.1 Herramienta seleccionada y por qué

La consigna ofrece Selenium o Cypress. Elegimos **Selenium WebDriver 4** con el ejecutor de pruebas nativo de Node (`node --test`):

Criterio	Selenium WebDriver	Cypress
Encaja con lo que ya usa el proyecto	Sí: mismo ejecutor (`node --test`) y mismo TypeScript que las 382 pruebas existentes	Trae su propio ejecutor, sintaxis y carpeta
Instalación	Paquete npm liviano; *Selenium Manager* descarga solo el driver del navegador instalado	Descarga un binario propio de ~500 MB, que con pnpm 11 obliga a habilitar scripts de instalación
Navegadores	Edge (desarrollo en Windows) y Chrome (CI) con el mismo código	Chromium, Firefox y WebKit
Varias sesiones en una misma prueba	Simple: dos navegadores, uno por tutor, en la tercera prueba	Limitado a una pestaña por prueba
Estándar	W3C WebDriver	API propia

Cypress tiene mejor experiencia de depuración (paso a paso visual). Pero para este stack —frontend sin framework servido por Express, y pruebas ya escritas con `node --test`— Selenium suma menos herramientas nuevas y corre igual en Windows y en el CI.

6.2 Configuración

- `web/backend/e2e/padres-solo-sus-hijos.e2e.ts`: las pruebas.
- `web/backend/scripts/test-e2e.ts`: levanta la pila completa (PostgreSQL, migraciones, seed, API y frontend) en un puerto libre y corre las pruebas con un navegador real en modo *headless*.
- `pnpm --filter backend test:e2e`: el comando.
- En el CI, job **Pruebas E2E (Selenium)**: PostgreSQL 18 como service container y Chrome del runner. Corre sólo en los PR, para no demorar el despliegue después del merge.

6.3 Ejemplo de prueba de regresión

Una **prueba de regresión** verifica que algo que ya funcionaba siga funcionando después de un cambio. La suite de RF-03 cumple ese rol: corre automáticamente en **cada PR**, sin importar qué módulo se haya tocado.

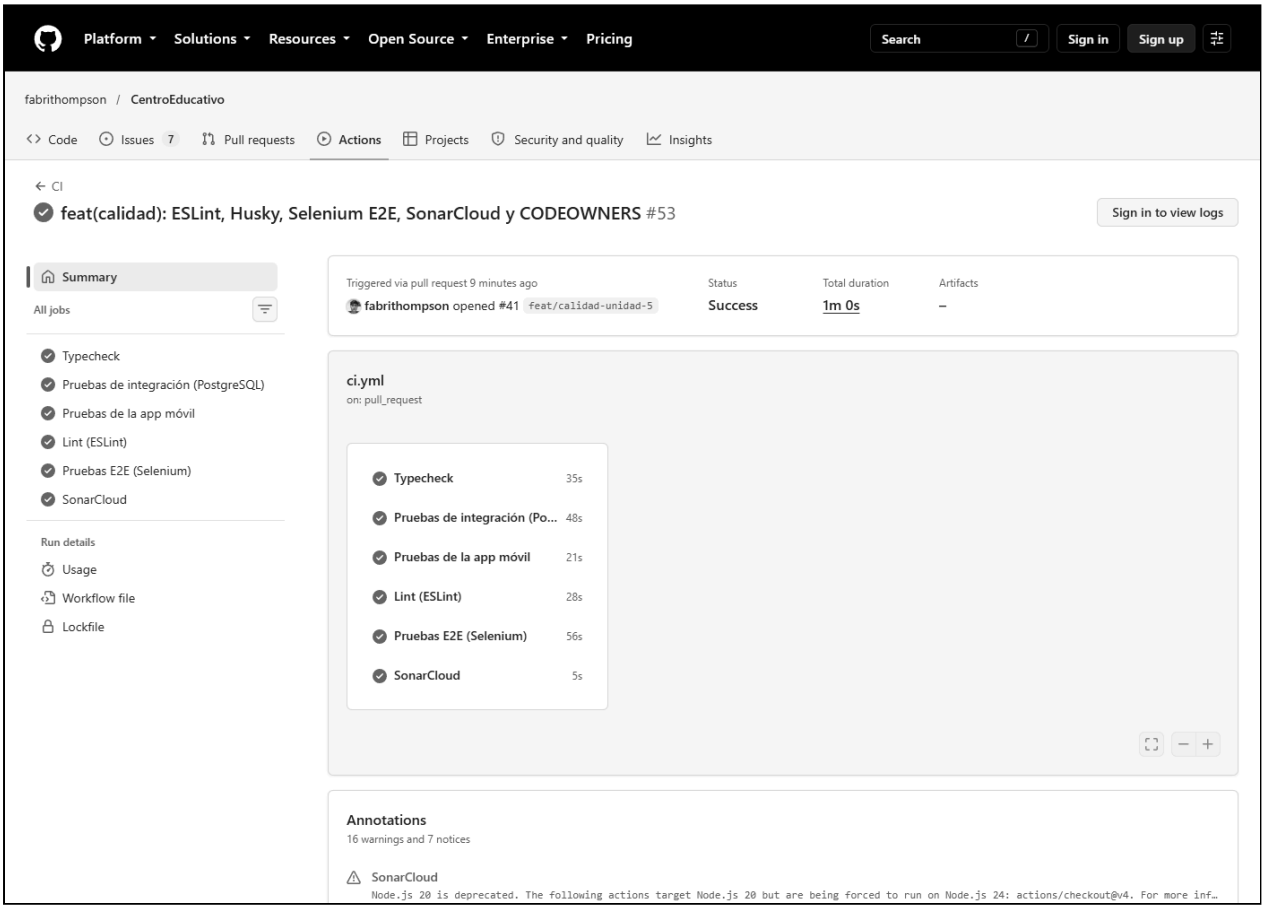
Escenario concreto: dentro de unos meses alguien agrega al panel de tutores un endpoint nuevo, por ejemplo `/api/padres/mis-hijos/:id/boletin`, y se olvida de colgarlo del middleware que valida el vínculo. El código compila, el lint pasa y las pruebas unitarias del boletín pasan, porque prueban el cálculo y no el acceso. Pero la funcionalidad que ya andaba (*un padre no ve a otros alumnos*) se rompió.

La prueba de regresión lo detecta antes de mergear: un tutor ajeno pide los datos de Mateo y, en lugar del 404 esperado, recibe 200. El job E2E queda en rojo, la protección de `main` bloquea el merge, y el defecto nunca llega a producción.

Las tres pruebas de la suite:

Prueba	**Qué verifica**
Una madre con dos hijos ve exactamente a esos dos	El selector muestra a María Paz Gómez e Ignacio Flores, y a nadie más
Una madre con un hijo ve sólo a ese hijo	El selector muestra sólo a Mateo Medina
No puede pedir los datos de un alumno que no es su hijo, aunque conozca el id	Con la sesión de otra madre, la API responde 404 a los deportes de Mateo

```
▶ RF-03 · un padre sólo ve a sus propios hijos
  ✓ una madre con dos hijos ve exactamente a esos dos
  ✓ una madre con un hijo ve sólo a ese hijo
  ✓ no puede pedir los datos de un alumno que no es su hijo, aunque
conozca el id
ℹ tests 3 · pass 3 · fail 0 · duration_ms 8459
E2E: TODO EN VERDE
```



CI del PR #41: el job «Pruebas E2E (Selenium)» junto con los otros cinco, todos en verde

7. Métricas

Elegimos dos métricas que las herramientas de los puntos anteriores ya producen, sin agregar infraestructura:

Métrica 1 — Densidad de problemas de análisis estático

Qué es: cantidad de problemas que encuentra el linter (errores + advertencias) por cada mil líneas de código (KLOC). Cuando SonarCloud esté activo, se complementa con sus *code smells*, bugs y la calificación de mantenibilidad.

Cómo se mide: `pnpm lint` en cada corrida del CI (job *Lint*). La cuenta por KLOC sale del reporte JSON de ESLint.

Momento	Errores	Advertencias	KLOC	Problemas / KLOC
Primer pase (26/10/2026)	36	27	≈ 32,1	≈ 1,96
Después de la adopción (26/10/2026)	**0**	27	32,4	**0,83**

Cómo se evalúa el rendimiento:

- **Errores: el umbral es 0, y es obligatorio.** El pre-commit y el CI ya lo garantizan; un error impide commitear y mergear.
- **Advertencias: tienen que bajar o, como mínimo, no subir.** Si un PR suma advertencias, se discute en la revisión. Meta para fin de año: menos de 0,5 por KLOC.
- **SonarCloud**: *quality gate* en verde sobre el código nuevo (sin bugs ni vulnerabilidades nuevas, duplicación menor al 3 %).

Métrica 2 — Tasa de éxito y tiempo de respuesta de la suite de regresión

Qué es: porcentaje de corridas de la suite E2E que pasan, y cuánto tarda en dar el resultado.

Cómo se mide: GitHub Actions registra cada corrida del job *Pruebas E2E (Selenium)* con su estado y duración; el script `scripts/metricas/entrega.mjs` lo resume.

Medición (26/10/2026)	Valor
Pruebas de la suite	3
Corridas en el CI	2 (PR #41 y #42) — **100 % exitosas**
Duración del job en el CI (con la pila completa)	56 s y 1 min 20 s
Duración de las pruebas en sí (local, Edge)	8,5 s

Cómo se evalúa el rendimiento:

- **Tasa de éxito en `main`: 100 %.** Una falla en un PR es buena noticia: la regresión se detectó antes de llegar a producción.

- **Pruebas inestables (*flaky*): 0.** Si una prueba falla y al reintentarla pasa sin cambios, se trata como un defecto de la prueba y se corrige. En este TP pasó dos veces durante el desarrollo de la suite (un selector que se regeneraba y una ventana demasiado chica), y ambas se corrigieron antes de integrarla.
- **Tiempo de respuesta: menos de 2 minutos.** Si la suite crece y lo supera, se paraleliza o se mueve a una corrida nocturna, manteniendo en cada PR el subconjunto crítico (RF-03 y pagos).